METODI MATEMATICI E STATISTICI PER LE DECISIONI ECONOMICHE

A.A. 2026/2027

Dizionario Python-R



Dizionario Python-R

In questo dizionario troverai la sintassi per Python ed R:

- **Cosa vuoi fare**: l'operazione da eseguire.
- **Come si scrive in Python**: il comando di riferimento.
- **Come si scrive in R**: utile se li conosci entrambi!

ALL'INTERNO

Gli strumenti
Valori e operazioni
Liste e dizionari
Decidere, ripetere, scrivere una funzione
Pacchetti e file
Guardare e selezionare i dati
Trasformare e riassumere i dati
Statistiche descrittive
I grafici

DALL'ESERCITAZIONE 1

Importare un dataset: da Colab e da Kaggle
Descrivere variabili qualitative e quantitative
Filtrare, creare colonne, unire tabelle
La regressione lineare

Gli strumenti

Come in R anche in Python prima di utilizzare i pacchetti bisogna importarli. Le librerie più utilizzate in questo corso sono:

SERVE A	LIBRERIA	SI IMPORTA CON
tabelle di dati	`polars`	`import polars as pl`
radici, logaritmi, numeri casuali	`math`, `random`	`import math`, `import random`
grafici	`plotnine`	`from plotnine import *`
modelli	`statsmodels`, `scikit-learn`	—

`plotnine` funziona come `ggplot2` di R: un grafico si costruisce aggiungendo pezzi con `+`. Se Colab non lo trova: `!pip install plotnine`.

Gli esempi usano un dataframe generico chiamato `dataframe`, letto da `file.csv`: `colonna`, `colonna1`, `colonna2` e `colonna3` sono colonne numeriche, `gruppo` è una colonna di categorie e `data` una colonna di date. Al posto di questi nomi metti quelli del tuo file.

```python
import math
import random
import polars as pl
from plotnine import *

x = [10, 20, 30, 40, 50]
l = {"a": 1, "b": 2}
dataframe = pl.read_csv("file.csv", try_parse_dates=True)
a = pl.DataFrame({"k": [1, 2, 3], "u": [1.0, 2.0, 3.0]})
b = pl.DataFrame({"k": [1, 2, 4], "v": [10.0, 20.0, 40.0]})
```

In R era:

```r
library(dplyr)
library(ggplot2)
library(readxl)

x <- c(10, 20, 30, 40, 50)
l <- list(a = 1, b = 2)
dataframe <- read.csv("file.csv")
a <- data.frame(k = c(1, 2, 3), u = c(1, 2, 3))
b <- data.frame(k = c(1, 2, 4), v = c(10, 20, 40))
```

Valori e operazioni

COSA VUOI FARE	PYTHON	IN R
assegnare un valore	`y = 5`	`y <- 5`
stampare	`print(y)`	`print(y)`
testo con dei valori dentro	`f"totale: {y}"`	`paste0("totale: ", y)`
vedere la classe	`type(y)`	`class(y)`
vero / falso	`True`, `False`	`TRUE`, `FALSE`
valore nullo	`None`	`NULL`
uguale, diverso	`y == 5`, `y != 5`	`y == 5`, `y != 5`
elevazione a potenza	`2 ** 3`	`2^3`
arrotondare	`round(3.14159, 2)`	`round(3.14159, 2)`
radice, esponenziale, logaritmo	`math.sqrt(16)`, `math.exp(1)`, `math.log(10)`	`sqrt(16)`, `exp(1)`, `log(10)`

Liste e dizionari

COSA VUOI FARE	PYTHON	IN R
una lista di valori	`[10, 20, 30]`	`c(10, 20, 30)`
contare quanti elementi	`len(x)`	`length(x)`
evidenziare il primo elemento	`x[0]`	`x[1]`
evidenziare l'ultimo elemento	`x[-1]`	`x[length(x)]`
evidenziare i primi tre elementi	`x[:3]`	`x[1:3]`
aggiungere un elemento come ultimo	`x.append(60)`	`x <- c(x, 60)`
controllare se un elemento è contenuto in	`30 in x`	`30 %in% x`
tra x1 e x2 (1:5)	`list(range(1, 6))`	`1:5`
somma, massimo, minimo	`sum(x)`, `max(x)`, `min(x)`	`sum(x)`, `max(x)`, `min(x)`
eseguire un'operazione su tutti i valori	`pl.Series(x) * 2`	`x * 2`
calcolare la media	`pl.Series(x).mean()`	`mean(x)`

COSA VUOI FARE	PYTHON	IN R
evidenziare le coppie nome/valore	`{"a": 1, "b": 2}`	`list(a = 1, b = 2)`
leggere un valore dal nome	`l["a"]`	`l$a`

ATTENZIONE

In Python si conta **da 0**: il primo elemento è `x[0]`.

Decidere, ripetere, scrivere una funzione

```python
voto = 21
if voto >= 18:
    esito = "Promosso"
else:
    esito = "Bocciato"

rendimenti = [0.01, -0.02, 0.03]
totale = 0
for r in rendimenti:
    totale = totale + r

def valore_futuro(capitale, tasso=0.02, anni=1):
    return capitale * (1 + tasso) ** anni
```

ATTENZIONE

Dopo aver fatto partire un ciclo di `if`, `else`, `for` e `def` ci vogliono i **due punti**, e le righe che stanno “dentro” si spostano a destra di quattro spazi. Per scrivere “altrimenti se” si scrive `elif`.

In R era:

```r
voto <- 21
if (voto >= 18) {
  esito <- "Promosso"
} else {
  esito <- "Bocciato"
}

rendimenti <- c(0.01, -0.02, 0.03)
totale <- 0
for (r in rendimenti) {
  totale <- totale + r
}

valore_futuro <- function(capitale, tasso = 0.02, anni = 1) {
  return(capitale * (1 + tasso)^anni)
}
```

Pacchetti e file

COSA VUOI FARE	PYTHON	IN R
installare un pacchetto	`!pip install pkg`	`install.packages("pkg")`
caricare un pacchetto	`import polars as pl`	`library(dplyr)`
leggere un CSV	`pl.read_csv("file.csv")`	`read.csv("file.csv")`

COSA VUOI FARE	PYTHON	IN R
CSV con una colonna di date	`pl.read_csv("file.csv", try_parse_dates=True)`	`dataframe$data <- as.Date(dataframe$data)`
CSV con `;` e virgola decimale	`pl.read_csv("file.csv", separator=";", decimal_comma=True)`	`read.csv2("file.csv")`
leggere un Excel	`pl.read_excel("file.xlsx")`	`read_excel("file.xlsx")`
salvare in CSV	`dataframe.write_csv("output.csv")`	`write.csv(dataframe, "output.csv", row.names = FALSE)`

> **NOTA**
> In Colab un file si carica dall'icona della cartella, a sinistra; sparisce quando chiudi la sessione. Se `read_excel` dà errore: `!pip install fastexcel`.

Guardare e selezionare i dati

Una tabella si chiama `DataFrame`. I comandi si scrivono **dopo il nome del dataframe, con un punto**: `dataframe.head()`.

COSA VUOI FARE	PYTHON	IN R
prime righe	`dataframe.head()`	`head(dataframe)`
quante righe e colonne	`dataframe.shape`	`dim(dataframe)`
nomi delle colonne	`dataframe.columns`	`names(dataframe)`
summary	`dataframe.describe()`	`summary(dataframe)`
evidenziare una colonna	`dataframe["colonna"]`	`dataframe$colonna`
indicare una colonna dentro un comando	`pl.col("colonna")`	`colonna` (dentro i comandi di `dplyr`)
evidenziare più colonne	`dataframe.select("colonna1", "colonna2")`	`select(dataframe, colonna1, colonna2)`
togliere una colonna	`dataframe.drop("colonna")`	`select(dataframe, -colonna)`
filtrare le righe	`dataframe.filter(pl.col("colonna1") > 35)`	`filter(dataframe, colonna1 > 35)`
filtrare con due condizioni	`dataframe.filter(pl.col("colonna1") > 35, pl.col("gruppo") == "valore")`	`filter(dataframe, colonna1 > 35, gruppo == "valore")`
contare per categoria	`dataframe["gruppo"].value_counts()`	`table(dataframe$gruppo)`
raggruppare le righe per categoria	`gruppi = dataframe.group_by("gruppo")`	`gruppi <- group_by(dataframe, gruppo)`
calcolare la media in ciascun gruppo	`gruppi.agg(media=pl.col("colonna").mean())`	`summarise(gruppi, media = mean(colonna))`

Trasformare e riassumere i dati

Trasformare significa modificare la struttura o i valori di una tabella; **riassumere** significa ricavare pochi valori da molte osservazioni, per esempio una media per ogni gruppo.

COSA VUOI FARE	PYTHON	IN R
creare una colonna (es. con il doppio dei valori di una esistente)	`dataframe.with_columns(nuova=pl.col("colonna") * 2)`	`mutate(dataframe, nuova = colonna * 2)`
rinominare una colonna	`dataframe.rename({"colonna": "nuovo_nome"})`	`rename(dataframe, nuovo_nome = colonna)`

COSA VUOI FARE	PYTHON	IN R
ordinare dal più grande	`dataframe.sort("colonna", descending=True)`	`arrange(dataframe, desc(colonna))`
contare quanti valori nulli	`dataframe["colonna"] .null_count()`	`sum(is.na(dataframe$colonna))`
togliere le righe con valori nulli	`dataframe.drop_nulls()`	`na.omit(dataframe)`
media per gruppo	`dataframe.group_by("gruppo") .agg(pl.col("colonna").mean())`	`dataframe %>% group_by(gruppo) %>% summarise(mean(colonna))`
unire due tabelle per una colonna comune	`a.join(b, on="k", how="left")`	`left_join(a, b, by = "k")`
mettere una tabella sotto l'altra	`pl.concat([a, a])`	`bind_rows(a, a)`

> **NOTA**
>
> In Polars `null` indica un valore mancante; `NaN` è un valore numerico speciale e non viene contato da `null_count()` né eliminato da `drop_nulls()`. In R `is.na()` riconosce sia `NA` sia `NaN`: i comandi nelle due colonne non sono quindi equivalenti in presenza di `NaN`.

Più passaggi uno dopo l'altro. Si mettono in fila con i punti, uno per riga; le parentesi esterne servono solo per poter andare a capo. In R si utilizza il `%>%`.

```python
(
    dataframe
    .filter(pl.col("colonna1") > 50)
    .group_by("gruppo")
    .agg(pl.col("colonna2").mean())
)
```

In R era:

```r
dataframe %>%
  filter(colonna1 > 50) %>%
  group_by(gruppo) %>%
  summarise(mean(colonna2))
```

Statistiche descrittive

SU UNA COLONNA

COSA VUOI FARE	PYTHON	IN R
media	`dataframe["colonna"].mean()`	`mean(dataframe$colonna)`
mediana	`dataframe["colonna"].median()`	`median(dataframe$colonna)`
minimo, massimo	`dataframe["colonna"].min()`, `dataframe["colonna"].max()`	`min(dataframe$colonna)`, `max(dataframe$colonna)`
deviazione standard	`dataframe["colonna"].std()`	`sd(dataframe$colonna)`
varianza	`dataframe["colonna"].var()`	`var(dataframe$colonna)`
quantile (es. primo quartile)	`dataframe["colonna"].quantile(0.25, interpolation="linear")`	`quantile(dataframe$colonna, 0.25)`
quanti sono negativi	`(dataframe["colonna"] < 0).sum()`	`sum(dataframe$colonna < 0)`
somma cumulata	`dataframe["colonna"].cum_sum()`	`cumsum(dataframe$colonna)`
rendimento composto	`(1 + dataframe["colonna"]) .product() - 1`	`prod(1 + dataframe$colonna) - 1`

ATTENZIONE

Per i quantili polars, se non glielo dici, prende il valore osservato più vicino; R invece fa una media tra i due valori vicini. Con `interpolation="linear"` i risultati coincidono con R. Vale anche per `describe()`: le righe `25%`, `50%` e `75%` tornano come in R con `dataframe.describe(interpolation="linear")`.

TRA PIÙ COLONNE

COSA VUOI FARE	PYTHON	IN R
covarianza tra due colonne	`dataframe.select(pl.cov("colonna1", "colonna2"))`	`cov(dataframe$colonna1, dataframe$colonna2)`
correlazione tra due colonne	`dataframe.select( pl.corr("colonna1", "colonna2"))`	`cor(dataframe$colonna1, dataframe$colonna2)`
matrice di correlazione	`dataframe.select("colonna1", "colonna2", "colonna3").corr()`	`cor(dataframe[, c("colonna1", "colonna2", "colonna3")])`

NUMERI CASUALI

COSA VUOI FARE	PYTHON	IN R
settare il seed per la replicabilità	`random.seed(1)`	`set.seed(1)`
estrarre un numero da una distribuzione normale	`random.gauss(0, 1)`	`rnorm(1)`
estrarre casualmente delle righe dal dataframe	`dataframe.sample(n=5, seed=1)`	`slice_sample(dataframe, n = 5)`

I grafici

Si parte da `ggplot(dati, aes(...))` e si aggiungono i pezzi con `+`. I nomi delle colonne vanno **tra virgolette**.

COSA VUOI FARE	PYTHON	IN R
istogramma	`ggplot(dataframe, aes(x="colonna")) + geom_histogram()`	`ggplot(dataframe, aes(x = colonna)) + geom_histogram()`
grafico a barre	`ggplot(dataframe, aes(x="gruppo")) + geom_bar()`	`ggplot(dataframe, aes(x = gruppo)) + geom_bar()`
boxplot per gruppo	`ggplot(dataframe, aes(x="factor(gruppo)", y="colonna")) + geom_boxplot()`	`ggplot(dataframe, aes(x = factor(gruppo), y = colonna)) + geom_boxplot()`
linea del tempo	`ggplot(dataframe, aes(x="data", y="colonna")) + geom_line()`	`ggplot(dataframe, aes(x = data, y = colonna)) + geom_line()`
fissare punti (due variabili)	`ggplot(dataframe, aes(x="colonna1", y="colonna2")) + geom_point()`	`ggplot(dataframe, aes(x = colonna1, y = colonna2)) + geom_point()`
assegnare colore in base ad un gruppo	`ggplot(dataframe, aes(x="colonna1", y="colonna2", color="gruppo")) + geom_point()`	`ggplot(dataframe, aes(x = colonna1, y = colonna2, color = gruppo)) + geom_point()`

PEZZI DA AGGIUNGERE

COSA VUOI FARE	PYTHON	IN R
titolo e nomi degli assi	`+ labs(title="Titolo", x="Asse x", y="Asse y")`	`+ labs(title = "Titolo", x = "Asse x", y = "Asse y")`
sfondo minimale	`+ theme_minimal()`	`+ theme_minimal()`
retta di regressione	`+ geom_smooth(method="lm")`	`+ geom_smooth(method = "lm")`
un grafico per gruppo	`+ facet_wrap("gruppo")`	`+ facet_wrap(~gruppo)`

Un grafico completo, con le parentesi per andare a capo:

```python
grafico = (
    ggplot(dataframe, aes(x="colonna1", y="colonna2", color="gruppo"))
    + geom_point()
    + labs(title="Titolo", x="Asse x", y="Asse y")
    + theme_minimal()
)
grafico                          # in Colab si vede da solo
grafico.save("grafico.png")      # per salvarlo
```

In R era:

```r
ggplot(dataframe, aes(x = colonna1, y = colonna2, color = gruppo)) +
  geom_point() +
  labs(title = "Titolo", x = "Asse x", y = "Asse y") +
  theme_minimal()
```

Importare un dataset: da Colab e da Kaggle

Un file dal tuo computer. In Colab clicca sull'icona della cartella, a sinistra, poi su *Carica* (la freccia verso l'alto) e scegli il file. Da lì si legge con il solo nome, `pl.read_csv("file.csv")`, come in *Pacchetti e file*.

COSA VUOI FARE	PYTHON	IN R

scaricare un dataset da Kaggle	`import kagglehub`
	`cartella = kagglehub`
	`.dataset_download(`
	`"utente/nome-dataset")`
		– (si scarica a mano dal sito di Kaggle)

vedere in che cartella è finito	`cartella`	–
leggere un file dentro quella cartella	`pl.read_csv(cartella + "/file.csv")`	`read.csv(file.path(cartella, "file.csv"))`
attaccare due testi	`"testo1" + "testo2"`	`paste0("testo1", "testo2")`

ATTENZIONE

Il nome da dare a `dataset_download` è la parte finale dell'indirizzo della pagina Kaggle: da `kaggle.com/datasets/mirichoi0218/insurance` si scrive `"mirichoi0218/insurance"`. Per i dataset pubblici non serve un account. Il file caricato a mano, invece, **sparisce quando chiudi Colab**: la volta dopo va ricaricato.

Descrivere variabili qualitative e quantitative

COME SI LEGGE

Il tipo è scritto sotto il nome della colonna, in `head()` e in `describe()`: `str` è testo (variabile **qualitativa**); `i64` sono numeri interi e `f64` numeri con la virgola (variabili **quantitative**); `bool` è vero/falso. Per le colonne `str`, `describe()` lascia vuote (`null`) media e deviazione standard.

COSA VUOI FARE	PYTHON	IN R
proporzioni per categoria (invece dei conteggi)	`dataframe["gruppo"] .value_counts(normalize=True)`	`prop.table( table(dataframe$gruppo))`
boxplot di una sola colonna	`ggplot(dataframe, aes(y="colonna")) + geom_boxplot()`	`ggplot(dataframe, aes(y = colonna)) + geom_boxplot()`
boxplot per gruppo (gruppo di testo)	`ggplot(dataframe, aes(x="gruppo", y="colonna")) + geom_boxplot()`	`ggplot(dataframe, aes(x = gruppo, y = colonna)) + geom_boxplot()`
dare un titolo alla legenda dei colori	`+ labs(color="Titolo")`	`+ labs(color = "Titolo")`

NOTA

Nel boxplot la scatola va dal primo al terzo quartile e la riga in mezzo è la mediana; i punti isolati sono i valori anomali. Se il gruppo è un numero (per esempio il numero di figli) si scrive `x="factor(gruppo)"`, come nella tabella dei grafici. La riga `50%` di `describe()` può differire di poco da `median()`: vedi il riquadro sui quantili in *Statistiche descrittive*.

Filtrare, creare colonne, unire tabelle

COSA VUOI FARE	PYTHON	IN R
tenere le righe di una categoria	`dataframe.filter( pl.col("gruppo") == "valore")`	`filter(dataframe, gruppo == "valore")`
le 5 righe con i valori più grandi	`dataframe.sort("colonna", descending=True).head(5)`	`head(arrange(dataframe, desc(colonna)), 5)`
creare una colonna vero/falso	`dataframe.with_columns( alto=pl.col("colonna") >= 30)`	`mutate(dataframe, alto = colonna >= 30)`
creare più colonne insieme	`dataframe.with_columns( nuova1=pl.col("colonna") / 12, nuova2=pl.col("colonna") >= 30)`	`mutate(dataframe, nuova1 = colonna / 12, nuova2 = colonna >= 30)`
media per due gruppi insieme	`dataframe.group_by("gruppo1", "gruppo2") .agg(pl.col("colonna").mean())`	`dataframe %>% group_by(gruppo1, gruppo2) %>% summarise(mean(colonna))`
scrivere una tabella a mano	`pl.DataFrame({"k": [1, 2], "v": ["a", "b"]})`	`data.frame(k = c(1, 2), v = c("a", "b"))`
una colonna calcolata da due colonne	`dataframe.with_columns( rapporto=pl.col("colonna1") / pl.col("colonna2"))`	`mutate(dataframe, rapporto = colonna1 / colonna2)`
la media di ogni gruppo, accanto a ogni riga	`medie = dataframe .group_by("gruppo").agg( media=pl.col("colonna") .mean()) dataframe.join(medie, on="gruppo", how="left")`	`medie <- dataframe %>% group_by(gruppo) %>% summarise(media = mean(colonna)) left_join(dataframe, medie, by = "gruppo")`

ATTENZIONE

`filter`, `sort` e `with_columns` **non cambiano** la tabella: mostrano il risultato e basta. Per tenere le colonne nuove bisogna riassegnare il nome: `dataframe = dataframe.with_columns(...)`. Nei confronti il testo va tra virgolette e si usa `==`, non `=`.

Unire due tabelle, un esempio. Una tabellina scritta a mano aggiunge a ogni riga il nome della zona in italiano; `join` cerca lo stesso valore di `region` nelle due tabelle.

```python
zone = pl.DataFrame({
    "region": ["northeast", "northwest", "southeast", "southwest"],
    "zona": ["Nord-Est", "Nord-Ovest", "Sud-Est", "Sud-Ovest"],
})
dataframe = dataframe.join(zone, on="region", how="left")
```

In R era:

```r
zone <- data.frame(
  region = c("northeast", "northwest", "southeast", "southwest"),
  zona = c("Nord-Est", "Nord-Ovest", "Sud-Est", "Sud-Ovest")
)
dataframe <- left_join(dataframe, zone, by = "region")
```

La regressione lineare

Il pacchetto è `statsmodels`: con `smf.ols` il modello si scrive **con la formula di R**, tra virgolette, prima la variabile da spiegare e poi `~` e i predittori. Negli esempi `colonna1` è la variabile da spiegare.

COSA VUOI FARE	PYTHON	IN R
caricare il pacchetto	`import statsmodels.formula.api as smf`	– (`lm` c'è già in R)
stimare una regressione semplice	`modello = smf.ols("colonna1 ~ colonna2", data=dataframe).fit()`	`modello <- lm(colonna1 ~ colonna2, data = dataframe)`
i coefficienti	`modello.params`	`coefficients(modello)`
un coefficiente (intercetta, pendenza)	`modello.params["Intercept"]`, `modello.params["colonna2"]`	`coefficients(modello)[1]`, `coefficients(modello)[2]`
la tabella completa	`modello.summary()`	`summary(modello)`
i p-value	`modello.pvalues`	`summary(modello)$coefficients[, 4]`
l'R^2	`modello.rsquared`	`summary(modello)$r.squared`
previsione a mano	`a + b * 50`	`coefficients(modello)[1] + 50 * coefficients(modello)[2]`
previsioni per nuovi valori	`nuovi = pl.DataFrame({"colonna2": [5, 10, 15]})` `modello.predict(nuovi)`	`predict(modello, newdata = data.frame(colonna2 = c(5, 10, 15)))`
previsioni con gli intervalli	`modello.get_prediction( {"colonna2": [5, 10, 15]}) .summary_frame(alpha=0.05)`	`predict(modello, newdata = data.frame(colonna2 = c(5, 10, 15)), interval = "confidence")`
regressione multipla	`smf.ols("colonna1 ~ colonna2 + colonna3", data=dataframe).fit()`	`lm(colonna1 ~ colonna2 + colonna3, data = dataframe)`
predittore qualitativo: una colonna 0/1	`dataframe = dataframe.with_columns( valore=(pl.col("gruppo") == "valore").cast(pl.Int8))` `smf.ols("colonna1 ~ valore", data=dataframe).fit()`	`dataframe$valore <- as.integer(dataframe$gruppo == "valore")` `lm(colonna1 ~ valore, data = dataframe)`

COSA VUOI FARE	PYTHON	IN R
la retta sopra i punti	`ggplot(dataframe, aes(x="colonna2", y="colonna1")) + geom_point() + geom_smooth(method="lm")`	`ggplot(dataframe, aes(x = colonna2, y = colonna1)) + geom_point() + geom_smooth(method = "lm")`
una retta per ogni gruppo	`ggplot(dataframe, aes(x="colonna2", y="colonna1", color="gruppo")) + geom_point() + geom_smooth(method="lm")`	`ggplot(dataframe, aes(x = colonna2, y = colonna1, color = gruppo)) + geom_point() + geom_smooth(method = "lm")`

LEGGERE IL RISULTATO

COSA	DOVE, NEL SUMMARY	COSA VUOL DIRE
intercetta (α)	riga `Intercept`, colonna `coef`	il valore previsto di y quando x vale 0
pendenza (β)	riga della x, colonna `coef`	di quanto cambia y, in media, se x aumenta di 1
p-value	colonna `P>|t|`	sotto 0.05 il coefficiente è significativamente diverso da zero: la x è significativa
R^2	`R-squared`, in alto a destra	la parte della variabilità della y spiegata dal modello, da 0 a 1
nella multipla	ogni riga della tabella	l'effetto di quella x **a parità delle altre**

COME SI LEGGE

In `summary()`: la colonna `coef` contiene i coefficienti, `P>|t|` i p-value, `R-squared` è in alto a destra. Per un predittore qualitativo si crea una colonna 0/1: vale 1 nelle righe della categoria `valore`, 0 nelle altre. Con più categorie si fa una colonna 0/1 per ognuna **tranne una**, che diventa la categoria di riferimento (per esempio `canale`: `negozio` e `sito`, e `App` è il riferimento). Il coefficiente di ogni colonna è quanto quella categoria si discosta, in media, dalla categoria di riferimento.

ATTENZIONE

In `pl.DataFrame` delle previsioni la colonna deve chiamarsi **esattamente come nel modello**. Un coefficiente molto piccolo (es. `0.0002`) non vuol dire per forza un effetto piccolo: dipende dall'unità di misura del predittore (un dollaro in più, non mille).